MAINCRAFTS TECHNOLOGY

VLSI Internship – Task 4 Report

RTL Design of Finite State Machines (FSM) and Control Units

Submitted by

Name	**:**	Vishesh Tyagi
College	**:**	Raj Kumar Goel Institute of Technology
Branch	**:**	Electronics and Computer Engineering
Internship	**:**	VLSI Internship
Task	**:**	Task 4

PART A :- Introduction to FSM

1. Introduction to FSM

A Finite State Machine (FSM) is a sequential digital circuit that operates by moving between a finite number of states based on input signals and clock pulses. The output of the circuit depends on the current state and, in some FSM types, also on the input. FSMs are widely used in digital systems for designing controllers and decision-making circuits.

2. Characteristics of FSM

- FSM contains a finite number of states.
- Only one state is active at a time.
- State transitions occur on the active clock edge.
- The next state depends on the present state and input.
- FSM is used in synchronous sequential circuits.

3. Basic Components of FSM

Component	Function
State Register	Stores the current state of the FSM.
Next State Logic	Determines the next state based on inputs.
Output Logic	Generates output according to FSM design.
Clock Signal	Synchronizes all state transitions.

4. Applications of FSM

- Traffic Light Controller
- Elevator Controller
- Vending Machine

5. Types of Finite State Machine (FSM)

1. Moore Machine :-

A Moore Machine is a type of FSM in which the output depends only on the present state of the machine. The output changes only when the state changes on the active clock edge.

Characteristics

- Output depends only on the current state.
- Output changes after the clock-triggered state transition.

2. Mealy Machine :-

A Mealy Machine is a type of FSM in which the output depends on both the present state and the current input. The output changes immediately when the input changes.

Characteristics

- Output depends on state as well as input.
- Faster response compared to Moore Machine.

PART B :- Moore FSM Design

Problem Statement

Design a simple Moore Finite State Machine (FSM) that cycles through three states in the sequence**: S0 → S1 → S2 → S0**

The output depends only on the current state of the FSM.

Theory

A Moore FSM is a sequential circuit in which the output is determined only by the present state. The output changes only after the state changes on the clock edge, making Moore machines stable and easy to design.

Design code

```
module moore_fsm(
  input clk,
  input reset,
  output reg [1:0]state );

  parameter S0 = 2'b00;
  parameter S1 = 2'b01;
  parameter S2 = 2'b10;

  always@(posedge clk)
begin

    if(reset)
      state <= S0;
    else
      S0 <= S1;
      S1 <= S2;
      S2 <= S0;
  default state : S0;

  end
endmodule
```

PART C :- Mealy FSM Design

Problem Statement

Output become HIGH when input = 1 and current state is S1.

Theory

A Mealy FSM is a sequential circuit where the output depends on the current state and the input. In this design, the output becomes HIGH (1) only when the FSM is in State S1 and input = 1.

Design code

```
module mealy_fsm(
   input clk,
   input reset,
   input in,
   output reg out );
reg state;
always @(posedge clk or posedge reset)
begin
   if (reset) begin
      state <= 0;
      out <= 0; end
 else begin

      case(state)
         0: begin
            if (in) begin
               state <= 1;
               out <= 0;
            end
         end

         1: begin
            if (in)
               out <= 1;
            else begin
               state <= 0;
               out <= 0;
                end
          end
     endcase
  end
end
endmodule
```

PART D :- Traffic Light Controller

Problem Statement

Design a Traffic Light Controller using a Finite State Machine (FSM) in Verilog HDL. The controller should switch the traffic lights in the sequence RED → GREEN → YELLOW → RED on every clock cycle and repeat continuously

Theory

A Traffic Light Controller is a real-world application of an FSM. It changes the traffic signal lights in a fixed sequence based on the clock signal. Each light represents a different state, and only one light remains ON at a time.

Design code

```
module traffic_light(
   input clk,
   input reset,
   output reg red,
   output reg yellow,
   output reg green );

reg [1:0] state;
parameter RED    = 2'b00,
            GREEN  = 2'b01,
            YELLOW = 2'b10;

   always @(posedge clk or posedge reset) begin
      if (reset)
         state <= RED;

      else begin
         case(state)
            RED:    state <= GREEN;
            GREEN:  state <= YELLOW;
            YELLOW: state <= RED;
            default: state <= RED;
         endcase
      end
   end
```

```
always @(*) begin
      red = 0;
      yellow = 0;
      green = 0;

      case(state)
         RED:    red = 1;
         GREEN:  green = 1;
         YELLOW: yellow = 1;
      endcase

   end
endmodule
```

PART E :- Sequence Detector (1011 Pattern Detector)

Objective

Detect binary sequence 1011 in serial input stream.

Design code

```
module sequence _detector(
input clk;
input reset;
input in;
output reg detected );
reg [2:0] state;
always @(posedge clk or posedge reset) begin
   if (reset) begin
      state <= 0;
      detected <= 0;
   end
else begin
      case(state)
         0: state <= (in) ? 1 : 0;
         1: state <= (in) ? 1 : 2;
         2: state <= (in) ? 3 : 0;
         3: begin
            if (in) begin
```

```
                detected <= 1;
                state <= 1;
            end else
                state <= 2;
        end
    endcase
  end
end

endmodule
```

PART F :- Testbench Development

1) Testbench for Moore FSM Design :-

```
module moore_fsm_tb();
  reg clk;
  reg reset;
  wire [1:0] state;
 moore_fsm dut (
     .clk(clk),
     .reset(reset),
     .state(state)  );

  always #5 clk = ~clk;

  initial begin
     clk = 0;
     reset = 1;

     #10;
     reset = 0;
    #60;
   $finish;
  end
initial begin
     $monitor ( "Time = %0t | Reset = %b | State = %b", $time, reset, state);
  end
endmodule
```

2) Testbench for Mealy FSM Design:-

```
module mealy_fsm_tb();
   reg clk;
   reg reset;
   reg in;
   wire out;

  mealy_fsm dut (
      .clk(clk),
      .reset(reset),
      .in(in),
      .out(out)  );

   always #5 clk = ~clk;

   initial begin
       clk = 0;
      reset = 1;
      in = 0;

      #10 reset = 0;

      #10 in = 1;
      #10  in = 1;
      #10 in = 0;
      #10 in = 1;
      #10 in = 1;
      #10 in = 0;

      #20 $finish;
   end

   initial begin
      $monitor ("Time=%0t Reset=%b In=%b Out=%b", $time, reset, in, out);
   end
endmodule
```

3) Testbench for Traffic Light Controller :-

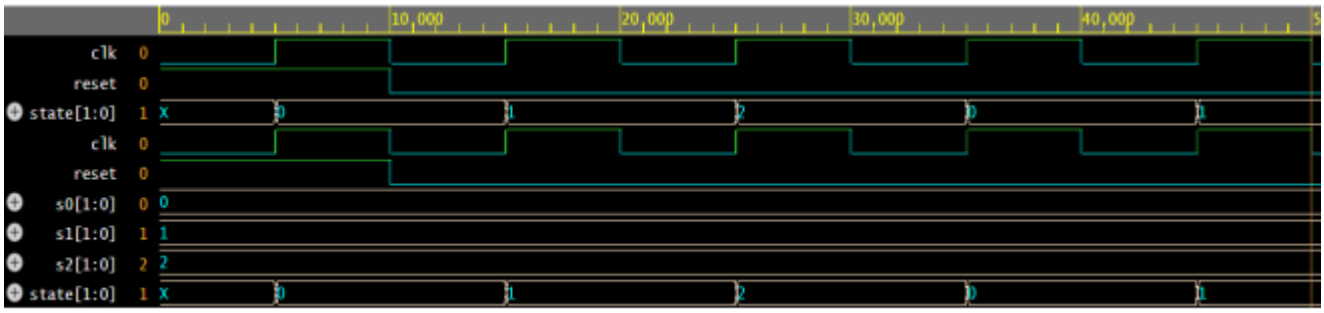
```
module traffic_light_tb;
  reg clk;
  reg reset;
  wire red;
  wire yellow;
  wire green;

  traffic_light dut (
     .clk(clk),
     .reset(reset),
     .red(red),
     .yellow(yellow),
     .green(green) );

  always #5 clk = ~clk;

  initial begin
     clk = 0;
     reset = 1;

     #10 reset = 0;

      #60;
  $finish;

  end

  initial begin
   $monitor ( "Time=%0t Reset=%b Red=%b Yellow=%b Green=%b",
            $time, reset, red, yellow, green);
  end
endmodule
```

4) Testbench for Sequence Detector :-

```
module sequence_detector_tb;
 reg clk;
 reg reset;
 reg in;
   wire detected;

   sequence_detector dut (
      .clk(clk),
      .reset(reset),
      .in(in),
      .detected(detected) );

   always #5 clk = ~clk;

   initial begin
      clk = 0;
      reset = 1;
      in = 0;

     #10 reset = 0;

      #10 in = 1;
      #10 in = 0;
      #10 in = 1;
      #10 in = 1;
      #10 in = 0;
      #10 in = 1;
      #10 in = 1;

       #20 $finish;

$monitor ( "Time=%0t Reset=%b In=%b Detected=%b", $time, reset, in,
detected);
   end
endmodule
```

PART G :- Simulation

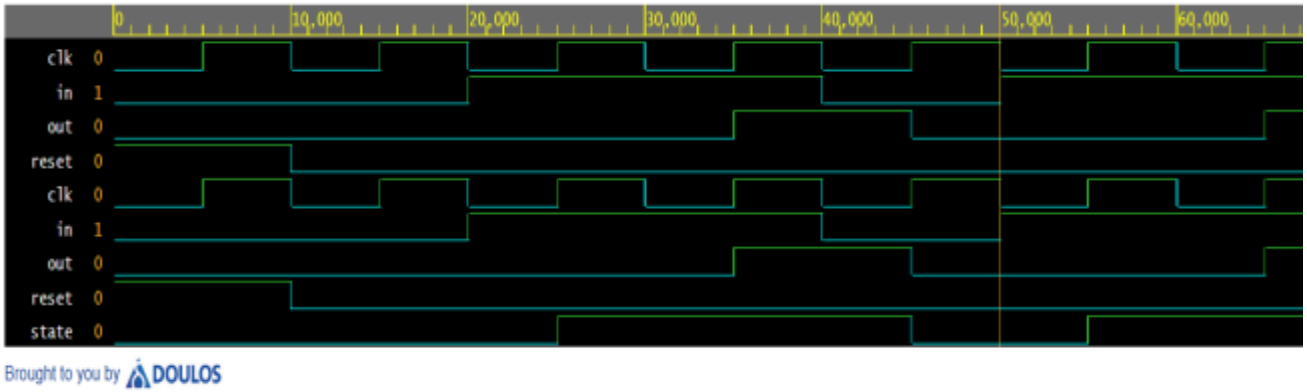
1. Waveform of Moore FSM



O/p of Moore FSM Design :-

Simulation Time (ns)	State	Meaning
0	xx	Initial unknown value before reset.
5	00	FSM enters S0 due to reset.
10	00	Reset released, state remains S0.
15	01	Transition from S0 → S1.
25	10	Transition from S1 → S2.
35	00	Transition from S2 → S0.

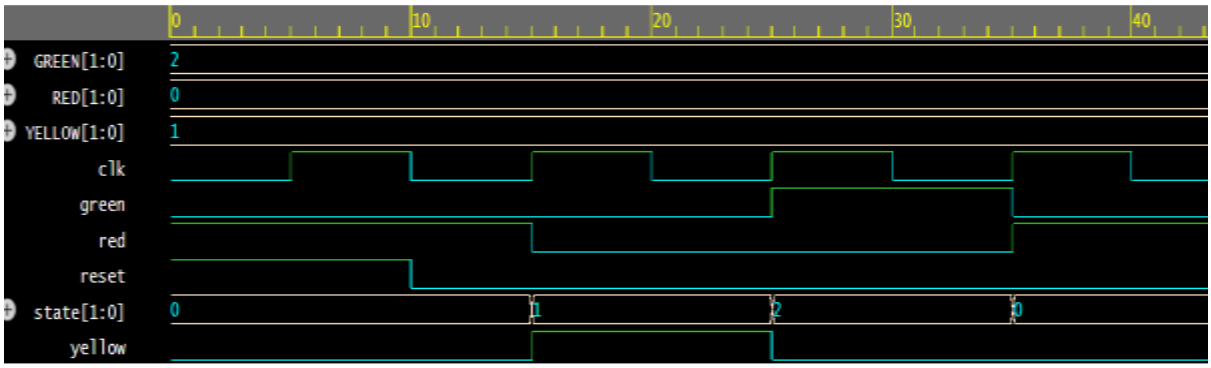
2. Waveform of Mealy FSM



O/p of Mealy FSM :-

Time (ns)	Reset	In	Out
0	1	0	0
10	0	0	0
20	0	1	0
30	0	1	1
40	0	0	0
50	0	1	0

3. Waveform of Traffic Light Controller

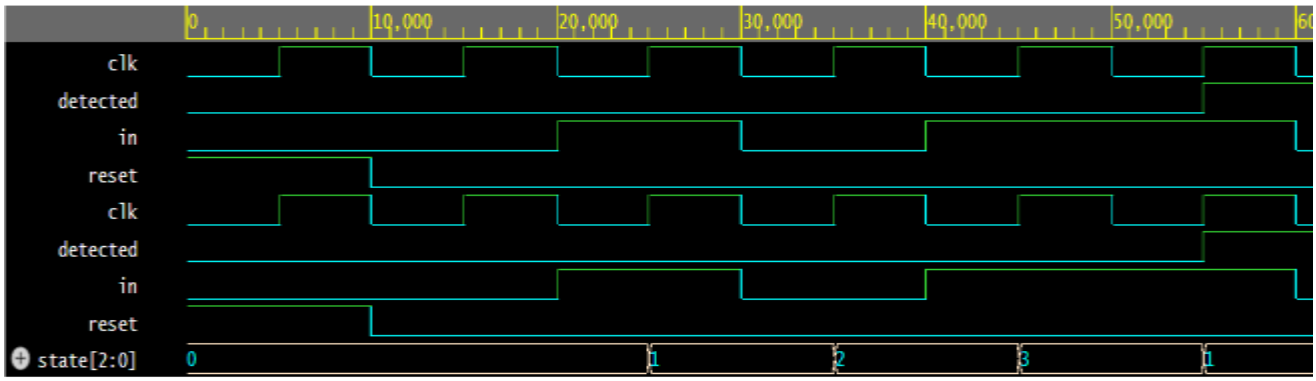


rought to you by DOULOS

O/p of Traffic Light Controller :-

Time (ns)	State
0	Red
10	Red
15	Yellow
25	Green
35	Red
45	Yellow(repeat)

4. Waveform of Sequence Detector



O/p of Sequence Detector :-

Time (ns)	Reset	In	Detected	State
0	1	0	0	S0
10	0	0	0	S0
20	0	1	0	S1
30	0	1	1	S2
40	0	0	0	S3
50	0	1	0	S1(Sequence 1011 Detected)

Conclusion

In this task, Finite State Machines (FSMs) were successfully studied, designed, and simulated using Verilog HDL. Moore and Mealy FSM implementations, Traffic Light Controller, and Sequence Detector were verified through simulation. The results confirmed correct state transitions and output behavior, providing a clear understanding of FSM-based RTL design and control unit implementation in VLSI systems.